〈リスト2-2〉モニタのプログラム・リスト

このリストは，CP/M68Kのアセンブラでアセンブルし，リンカにより実際のアドレスに配置しているため，実際のアドレスとは異なる．リストのアドレスに$ＦＦ００００を加えた値が実際のアドレスであり，ワーク・エリア(ライン１０００以降)は$４００を加えた値が実際のワーク・アドレスとなる．

リストに示すオペランド値も，ワーク・エリアを使用している箇所は$４００を加えた値が実際の値であり，ベクタ・テーブルのデータ(ライン８４５〜８７８)は実際の値では$ＦＦ００００が加わる．

ROMの作成はダンプ・リスト(**リスト2-1**)を使用のこと．

```
D>C P / M   6 8 0 0 0   A s s e m b l e rRevision 02.01Page   1
Source File: m:mon13.s

  1
  2                                  ****** 68000 monitor ******
  3                                  *                April 1984 *
  4                                  ****************************
  5
  6                                  aciac   = $ffff01 シリアル・ポートのアドレス
  7                                  pitadr  = $fff001 パラレル・ポート/タイマのアドレス
  8                                                                  アセンブラに対
  9                                  stkadr  = $800    スタック・ポインタの初期化  しての定義
 10                                  aciad   = aciac+2
 11
 12
 13                                  *** cold start default ***
 14 00000000 00000800                        dc.l    stkadr  スタック・アドレス
 15 00000004 00000008                        dc.l    resjmp  実行開始アドレス  リセット・スタート・ベクタ
 16
 17
 18                                  *** i/o initialize ***
 19 00000008 13FC000300FFFF01        resjmp: move.b  #$03,aciac
 20 00000010 13FC001100FFFF01                move.b  #$11,aciac  シリアル・ポートのイニシャライズ
 21 00000018 207C00000800                    move.l  #stkadr,a0  8ビット,2ストップ・ビット,パリティなし
 22 0000001E 4E60                            move.l  a0,usp          usp inz
 23 00000020 61000602                        bsr     vecinz          vector inz.
 24 00000024 600009DE                        bra     inz2            intz printer
 25
 26                                  *** input key to d0 reg. ***
 27 00000028 0839000000FFFF01        inkey:  btst    #0,aciac    1文字入力ルーチン
 28 00000030 67F6                            beq     inkey
 29 00000032 103900FFFF03                    move.b  aciad,d0
 30 00000038 4E75                            rts
 31
 32                                  *** scan key to zero flag. ***
 33 0000003A 0839000000FFFF01        sckey:  btst    #0,aciac    キーが打たれたかどうかのチェック.打たれ
 34 00000042 6708                            beq     scret       ていれば,データをD0に入れてリターン,打た
 35 00000044 103900FFFF03                    move.b  aciad,d0    れてなければ,D0をクリアしてリターン
 36 0000004A 4E75                            rts
 37 0000004C 4200                    scret   clr.b   d0
 38 0000004E 4E75                            rts
 39
 40                                  *** output byte console from d0 reg; ***
 41 00000050 48E7C000                outbyt: movem.l d0/d1,-(a7)     D0の下位1バイトを16進表示
 42 00000054 1200                            move.b  d0,d1
 43 00000056 E810                            roxr.b  #4,d0
 44 00000058 610A                            bsr     outdgt
 45 0000005A 1001                            move.b d1,d0
 46 0000005C 6106                            bsr     outdgt
 47 0000005E 4CDF0003                        movem.l (a7)+,d0/d1
 48 00000062 4E75                            rts
 49
 50                                  *** one-digit  output from d0 reg ***
 51 00000064 0200000F                outdgt: andi.b #$0f,d0      D0の下位4ビットを
 52 00000068 0C00000A                        cmpi.b  #$0a,d0         16進1文字表示
 53 0000006C 6502                            bcs     dgtskp
 54 0000006E 5E00                            addi.b  #$7,d0
 55 00000070 06000030                dgtskp: addi.b  #$30,d0
 56 00000074 600009F0                outvd1: bra     outchk
 57
 58                                  *** output word to console from d0 reg. ***
 59 00000078 48E7C000                outwor: movem.l d0/d1,-(a7)     D0の下位1ワードを16進表示
 60 0000007C 3200                            move.w  d0,d1
 61 0000007E E050                            roxr.w  #8,d0
 62 00000080 61CE                            bsr     outbyt
 63 00000082 3001                            move.w  d1,d0
 64 00000084 61CA                            bsr     outbyt
 65 00000086 4CDF0003                        movem.l (a7)+,d0/d1
 66 0000008A 4E75                            rts
 67
 68                                  *** output long word to console from d0 reg. ***
 69 0000008C 48E7C000                outlon: movem.l d0/d1,-(a7)     D0の全データ,2ワードを16進表示
 70 00000090 3200                            move.w  d0,d1
 71 00000092 4840                            swap    d0
 72 00000094 61E2                            bsr     outwor
 73 00000096 3001                            move.w  d1,d0
 74 00000098 61DE                            bsr     outwor
 75 0000009A 4CDF0003                        movem.l (a7)+,d0/d1
 76 0000009E 4E75                            rts
 77
 78                                  *** output 3-bytes and space from a0 reg. ***
 79 000000A0 48E7C000                outa0s: movem.l d0/d1,-(a7) A0の下位3バイトを16進出力し,スペー
 80 000000A4 2008                            move.l  a0,d0           スを付加する.
 81 000000A6 3208                            move.w  a0,d1
 82 000000A8 4840                            swap    d0              アドレス値の表示に使用される
 83 000000AA 61A4                            bsr     outbyt
 84 000000AC 3001                            move.w  d1,d0
 85 000000AE 61C8                            bsr     outwor
 86 000000B0 6106                            bsr     outsp
 87 000000B2 4CDF0003                        movem.l (a7)+,d0/d1
 88 000000B6 4E75                            rts
 89
 90                                  *** output space code ***
 91 000000B8 2F00                    outsp:  move.l  d0,-(a7)        スペース・コードの出力
 92 000000BA 7020                            moveq   #$20,d0
 93 000000BC 61B6                    outcd:  bsr     outvd1
 94 000000BE 201F                            move.l  (a7)+,d0
 95 000000C0 4E75                            rts
 96
 97                                  *** output error code ***
 98 000000C2 2F00                    outer:  move.l  d0,-(a7)            ?コードの出力
 99 000000C4 703F                            moveq   #$3f,d0         '?'
100 000000C6 60F4                            bra     outcd
101
102                                  *** output carriage return code ***
103 000000C8 2F00                    outcr:  move.l  d0,-(a7)    ライン・フィード,キャリッジ・リターンの出力
104 000000CA 700D                            moveq   #$0d,d0
105 000000CC 61A6                            bsr     outvd1
```

```
000000CE 700A                moveq   #$0a,d0
000000D0 60EA                bra outcd

                     *** output asterisk code ***
000000D2 2F00        outast: move.l  d0,-(a7)     アスタリスク"*"の表示
000000D4 702A                moveq   #$2a,d0          '*'
000000D6 60E4                bra     outcd

                     *** output word of d0 reg and space ***
000000D8 619E        outws:  bsr     outwor           D0の下位1ワードを16進出力
000000DA 60DC                bra     outsp            し,スペースを付加する

                     *** input and echoback ***
000000DC 6100FF4A    inecho: bsr     inkey            エコー・バック付き1文字入力
000000E0 60000984            bra     outchk


                     *** input byte to d0 reg. ***
000000E4 2F01        inbyte: move.l  d1,-(a7)     ターミナルから1バイト(16進数2文字)を
000000E6 610C                bsr     inhex            入力し,D0の下位1バイトに入れてリター
000000E8 E900                asl.b   #4,d0            ン.
000000EA 1200                move.b  d0,d1            キー入力はエコー・バックされる
000000EC 6106                bsr     inhex
000000EE 8001                or.b    d1,d0
000000F0 221F                move.l  (a7)+,d1
000000F2 4E75                rts

000000F4 61E6        inhex:  bsr     inecho
000000F6 04000030            subi.b  #$30,d0
000000FA 65F8                bcs     inhex
000000FC 0C00000A            cmpi.b  #$0a,d0
00000100 650A                bcs     inhexe
00000102 04000027            subi.b  #$27,d0
00000106 0C000010            cmpi.b  #$10,d0
0000010A 64E8                bcc     inhex
0000010C 4E75        inhexe: rts


                     *** input word to d0 reg. ***
0000010E 2F01        inword: move.l  d1,-(a7)     ターミナルから1ワードを入力し,D0の下位
00000110 61D2                bsr     inbyte           1ワードに入れてリターン.
00000112 E140                asl.w   #8,d0            キー入力はエコー・バックされる
00000114 3200                move.w  d0,d1
00000116 61CC                bsr     inbyte
00000118 8041                or.w    d1,d0
0000011A 221F                move.l  (a7)+,d1
0000011C 4E75                rts


                     *** output space and input address to a0 reg. ***
0000011E 2F01        inspa0: move.l  d1,-(a7)
00000120 6196                bsr     outsp
00000122 7024                moveq   #$24,d0          '$'   スペースを出力し,ターミ
00000124 61000940            bsr     outchk               ナルから 3バイトを入力
00000128 61BA                bsr     inbyte               してリターン.
0000012A 4840                swap    d0                   データはA0に入る.
0000012C 2200                move.l  d0,d1                エコー・バック付き.
0000012E 61DE                bsr     inword               アドレス値の入力に使用
00000130 8081                or.l    d1,d0                される
00000132 08800000            bclr    #0,d0
00000136 2040                movea.l d0,a0
00000138 221F                move.l  (a7)+,
0000013A 4E75                rts


                     *** input double address to a0 and a1 ***
0000013C 61E0        inarea: bsr     inspa0       ターミナルから,二つのアドレス値を入力.
0000013E 2248                movea.l a0,a1        最初のアドレス値はA0,2番目のアドレス
00000140 61DC                bsr     inspa0       値はA1に入る
00000142 C149                exg     a0,a1
00000144 4E75                rts


                     *** output carriage return and a0 reg. ***
00000146 6180        outcra: bsr     outcr        ライン・フィード,キャリッジ・リターンを
00000148 6000FF56            bra     outa0s       出力し,A0の3位3バイトを表示


                     *** output massage to console from a0   reg. indirect ***
0000014C 2F00        outmsg: move.l  d0,-(a7)     A0をポインタとし,ポインタの示す文字列
0000014E 1018        msglop: move.b  (a0)+,d0     をターミナルに表示.
00000150 6706                beq     msgend       文字列の終了マークは$00
00000152 61000912            bsr     outchk
00000156 60F6                bra     msglop
00000158 201F        msgend: move.l  (a7)+,d0
0000015A 4E75                rts


                     *** check and convert hex-code ***
0000015C 04000030    chkhex: subi.b  #$30,d0      D0の16進文字コードをバイナリ・コードに
00000160 6516                bcs     hexerr       変換.
00000162 0C00000A            cmpi.b  #$0a,d0      エラーがなければゼロ・フラグをセット,
00000166 650A                bcs     hexend       エラーがあればゼロ・フラダをクリアし
00000168 04000027            subi.b  #$27,d0      てリターン
0000016C 0C000010            cmpi.b  #$10,d0
00000170 6406                bcc     hexerr
00000172 003C0004    hexend: ori.b   #$04,ccr
00000176 4E75                rts
00000178 023C001B    hexerr: andi.b  #$1b,ccr
0000017C 4E75                rts


                     *** output memory block dump ***
0000017E 48E7C000    outmb:  movem.l d0/d1,-(a7)  A0で示すアドレスから,D0で示すワード
00000182 3200                move.w  d0,d1        ド数だけメモリ内容をダンプ表示.
00000184 3018        mbloop: move.w  (a0)+,d0     出力はワード単位にスペースで区切られ
00000186 6100FF50            bsr     outws        る.
0000018A 5341                sub.w   #1,d1        A0は出力したワード数の2倍だけ進む.
0000018C 66F6                bne     mbloop       (注) 出力データはワード単位,アドレス
0000018E 4CDF0003            movem.l (a7)+,d0/d1      値はバイト単位
00000192 4E75                rts


                     *** output memory character dump ***
00000194 48E7C000    outchr: movem.l d0/d1,-(a7)  A0で示すアドレスから,D0で示すバイト
00000198 3200                move.w  d0,d1        数だけアスキ・ダンプ.
0000019A 1018        chrlop: move.b  (a0)+,d0     $20未満および$7F以上はピリオドを
0000019C 0C000020            cmpi.b  #$20,d0      表示する
000001A0 6506                bcs     chskp1
000001A2 0C00007E            cmpi.b  #$7e,d0
000001A6 6302                bls     chrskp
000001A8 702E        chskp1  moveq   #$2e,d0
000001AA 610008BA    chrskp: bsr     outchk
000001AE 5341                subq.w  #1,d1
000001B0 66E8                bne     chrlop
000001B2 4CDF0003            movem.l (a7)+,d0/d1
000001B6 4E75                rts


                     *** check break ***
000001B8 2F00        chkbrk: move.l  d0,-(a7)     キーが打たれているかをチェック.打たれてな
000001BA 6100FE7E            bsr     sckey        ければゼロ・フラグをセットしてリターン.
000001BE 6710                beq     ckbend       打たれていればスペース・コードであるかを調べ
```

〈リスト2-2〉モニタ（つづき）

```
000001C0 0C000020          cmpi.b  #$20,d0   る．スペース・コードであれば，次に1文字打たれる
000001C4 660A              bne     ckbend    まで待ち，そのコードが再びスペース・コードであ
000001C6 6100FE72  brklp:  bsr     sckey     か調べスペース・コードであればゼロ・フラグをセ
000001CA 67FA              beq     brklp     ット．スペース・コードでなければ，ゼロ・フラグを
000001CC 0C000020          cmpi.b  #$20,d0   クリアしてリターン
000001D0 4CDF0001  ckbend: movem.l (a7)+,d0
000001D4 4E75              rts

                   *** output byte, space, memory block long word ***
000001D6 6100FE78  outbsm: bsr     outbyt

                   *** output memoty block 4-long words ***
000001DA 48E7C000  outmlg: movem.l d0/d1,-(a7) A0で示すメモリ・ブロックを四つのロング・
000001DE 7204              moveq   #4,d1       ワードとしてターミナルへ出力．
000001E0 6100FED6  mlglp:  bsr     outsp       A0は16バイト進む．
000001E4 2018              move.l  (a0)+,d0    レジスタ・ダンプに使用される
000001E6 6100FEA4          bsr     outlon
000001EA 5341              subq.w  #1,d1
000001EC 66F2              bne     mlglp
000001EE 4CDF0003  out6sk: movem.l (a7)+,d0/d1      from out6by
000001F2 4E75              rts

                   *** display registers ***
000001F4 48E7FFFF  outreg: movem.l d0-d7/a0-a7,-(a7) レジスタ・ダンプ
000001F8 4E68      move.l  usp,a0
000001FA 2008              move.l  a0,d0
000001FC 6100FEBA          bsr     outsp       ユーザ・スタック・ポインタの表示
00000200 6100FE8A          bsr     outlon
00000204 6100FEC2          bsr     outcr
00000208 204F              movea.l a7,a0
0000020A 70D0              moveq   #$d0,d0     D0～D3の表示
0000020C 61C8              bsr     outbsm
0000020E 611C              bsr     outdsp
00000210 70D4              moveq   #$d4,d0
00000212 61C2              bsr     outbsm      D4～D7の表示
00000214 6100FEB2          bsr     outcr
00000218 70A0              moveq   #$a0,d0
0000021A 61BA              bsr     outbsm      A0～A3の表示
0000021C 610E              bsr     outdsp
0000021E 70A4              moveq   #$a4,d0
00000220 61B4              bsr     outbsm      A4～A7の表示
00000222 6100FEA4          bsr     outcr
00000226 4CDFFFFF          movem.l (a7)+,d0-d7/a0-a7
0000022A 4E75              rts

                   *** output double spaces ***
0000022C 6100FE8A  outdsp: bsr     outsp
00000230 6000FE86          bra     outsp

                   *** output binary word of d0 reg. ***
00000234 48E7C000  outbiw: movem.l d0/d1,-(a7)     D0の下位1ワードを2進数で表示
00000238 610008B8          bsr     outbx

0000023C 6100FE7A          bsr     outsp
00000240 3001              move.w  d1,d0
00000242 6106              bsr     outbin
00000244 4CDF0003          movem.l (a7)+,d0/d1
00000248 4E75              rts

                   *** output binary byte of d0 reg. ***
0000024A 48E7E000  outbin: movem.l d0-d2,-(a7)     D0の下位1バイトを2進数で表示
0000024E 3200              move.w  d0,d1
00000250 7408              moveq   #8,d2
00000252 7030      binlop: moveq   #$30,d0
00000254 E301              asl.b   #1,d1
00000256 6402              bcc     binskp
00000258 5200              addq.b  #1,d0
0000025A 6100080A  binskp: bsr     outchk
0000025E 5302              subq.b  #1,d2
00000260 66F0              bne     binlop
00000262 4CDF0007          movem.l (a7)+,d0-d2
00000266 4E75              rts

                   *** check input carriage return or not ***
00000268 48E78000  chkcr:  movem.l d0,-(a7)        スペース表示し，エコー・バッ
0000026C 6100FE4A          bsr     outsp           クなしの入力を行う．
00000270 702D              moveq   #$2d,d0     '-' 入力されたコードがキャリッ
00000272 6100FDB4          bsr     inkey           ジ・リターンであれば，"！"を
00000276 0C00000D          cmpi.b  #$0d,d0         表示し，ゼロ・フラグをセット
0000027A 660A              bne     cr2skp          してリターン．キャリッジ・
0000027C 7021              moveq   #$21,d0     '!' リターンでなければゼロ・フラ
0000027E 610007E6          bsr     outchk          グはクリアされてリターン
00000282 003C0004          ori.b   #04,ccr     set zero flag
00000286 4CDF0001  cr2skp: movem.l (a7)+,d0
0000028A 4E75              rts

                   *** output 3-bytes of d0 reg. ***
0000028C 48E7C000  out3by: movem.l d0/d1,-(a7)     "－－"を表示し，D0の下位
00000290 2200              move.l  d0,d1           3バイトを16進表示．
00000292 702D              moveq   #$2d,d0     '-' プログラム・カウンタの表示
00000294 610007D0          bsr     outchk          サブルーチン
00000298 610007CC          bsr     outchk
0000029C 2001              move.l  d1,d0
0000029E 4840              swap    d0
000002A0 6100FDAE          bsr     outbyt
000002A4 3001              move.w  d1,d0
000002A6 6100FDD0          bsr     outwor
000002AA 6000FF42          bra     out6sk          (movem & rts)

                   *** memory examine and change ***  ワード単位でのメモリ・データの読み出しおよび
                                                      書き換え（mコマンドで起動する）
000002AE 6100FE6E  memory: bsr     inspa0  アドレスの入力
000002B2 61B4              bsr     chkcr   キャリッジ・リターンかどうかのチェック
000002B4 662A              bne     mend    キャリッジ・リターンでなければ終了
000002B6 6100FE8E  mnext:  bsr     outcra
000002BA 3018              move.w  (a0)+,d0
000002BC 6100FDBA          bsr     outwor
000002C0 6100FDF6  mcom:   bsr     outsp
000002C4 6100FE16          bsr     inecho
000002C8 0C000020          cmpi.b  #$20,d0     スペース・コードの入力で次のアドレス
000002CC 67E8              beq     mnext
000002CE 0C00002F          cmpi.b  #'/',d0     "/"ではアドレスを1ワードもどす
000002D2 6748              beq     mback
000002D4 0C00002C          cmpi.b  #',',d0     ","ではデータの書き換え
000002D8 672E              beq     mchang
000002DA 0C00002E          cmpi.b  #'.',d0     "."では終了
000002DE 6602              bne     mskp
000002E0 4E75      mend:   rts
000002E2 6100FE78  mskp:   bsr     chkhex
000002E6 66D8              bne     mcom
000002E8 2F01      mchngl: move.l  d1,-(a7)    メモリ・データの書き換え
000002EA E900              asl.b   #4,d0
000002EC 1200              move.b  d0,d1
000002EE 6100FDEC  mcloop: bsr     inecho
```

## 〈リスト2-2〉モニタ（つづき）

```
000002F2 6100FE68          bsr     chkhex
000002F6 66F6              bne     mcloop
000002F8 8001              or.b    d1,d0
000002FA E140              asl.w   #8,d0
000002FC 3200              move.w  d0,d1
000002FE 6100FDE4          bsr     inbyte
00000302 8041              or.w    d1,d0
00000304 221F              move.l  (a7)+,d1
00000306 6004              bra     mchng2
00000308 6100FE04  mchang: bsr     inword
0000030C 3140FFFE  mchng2: move.w  d0,-2(a0)
00000310 B068FFFE          cmp.w   -2(a0),d0
00000314 67AA              beq     mcom
00000316 6100FDAA          bsr     outer
0000031A 60A4              bra     mcom
0000031C 41E8FFFC  mback:  lea     -4(a0),a0
00000320 6094              bra mnext

                                                   1行を40文字としたメモリ・
                           *** print memory 40 character *** ダンプ（pコマンドで起動する）
00000322 6100FE18  print:  bsr     inarea   先頭アドレス,終了アドレスの入力
00000326 6100FF40          bsr     chkcr
0000032A 6622              bne     pend
0000032C 2448      prnlp:  movea.l a0,a2
0000032E 7004              moveq   #4,d0
00000330 6100FE14          bsr     outcra   4ワードを16進ダンプ
00000334 6100FE48          bsr     outmb
00000338 6100FD7E          bsr     outsp
0000033C 204A              movea.l a2,a0
0000033E 7008              moveq   #8,d0    8バイトをアスキ・ダンプ
00000340 6100FE52          bsr     outchr
00000344 6100FE72          bsr     chkbrk
00000348 6604              bne     pend
0000034A B3C8              cmpa.l  a0,a1
0000034C 64DE              bcc     prnlp
0000034E 4E75      pend:   rts

                           *** wide print memory 80 characters *** 1行を80文字とした
                                                                   メモリ・ダンプ
00000350 6100FDEA  wide:   bsr     inarea   （wコマンドで起動する）
00000354 6100FF12          bsr     chkcr
00000358 6622              bne     wend
0000035A 2448      wloop:  movea.l a0,a2
0000035C 7008              moveq   #8,d0
0000035E 6100FDE6          bsr     outcra   8ワードを16進ダンプ
00000362 6100FE1A          bsr     outmb
00000366 6100FD50          bsr     outsp
0000036A 204A              movea.l a2,a0
0000036C 7010              moveq   #$10,d0  16バイトをアスキ・ダンプ
0000036E 6100FE24          bsr     outchr
00000372 6100FE44          bsr     chkbrk
00000376 6604              bne     wend
00000378 B3C8              cmpa.l  a0,a1
0000037A 64DE              bcc     wloop
0000037C 4E75      wend:   rts

                                                   ブランチ命令のオフセット値を計算して
                           *** branch caliculate *** 表示（oコマンドで起動する）
0000037E 6100FDBC  branch: bsr     inarea   ブランチ命令の置かれているアドレス,
00000382 6100FEE4          bsr     chkcr    およびブランチ先のアドレス値を入力
00000386 661A              bne     bend
00000388 2009              move.l  a1,d0
0000038A 9088              sub.l   a0,d0
0000038C 5580              subq.l  #2,d0
0000038E 0C8000008000      cmpi.l  #$8000,d0
00000394 650E              bcs     bran1
00000396 0C8000FF8000      cmpi.l  #$ff8000,d0 ブランチ可能な範囲かどうかを調べる.
00000398 6406              bcc     bran1       ブランチできない範囲であれば"?"を
0000039E 6100FD22          bsr     outer       表示してリターン
000003A2 4E75      bend:   rts
000003A4 1210      bran1:  move.b  (a0),d1
000003A6 020100F0          andi.b  #$f0,d1
000003AA 0C010060          cmpi.b  #$60,d1
000003AE 660C              bne     bword
000003B0 0C400080          cmpi.w  #$0080,d0
000003B4 651A              bcs     bbyte       8ビットのオフセット値で可能かを調べ,
000003B6 0C40FF80          cmpi.w  #$ff80,d0   可能であれば,バイト・オフセットとする
000003BA 6414              bcc     bbyte
000003BC 6100FCFA  bword:  bsr     outsp       ワード・オフセット
000003C0 6100FCB6          bsr     outwor      ワード・オフセット値を表示
000003C4 6100FEA2          bsr     chkcr
000003C8 66D8              bne     bend        キャリッジ・リターンが打たれた場合は,
000003CA 31400002          move.w  d0,+2(a0)   オフセット値をメモリに書き込む
000003CE 60D2              bra     bend
000003D0 6100FCE6  bbyte:  bsr     outsp       バイト・オフセット
000003D4 6100FC7A          bsr     outbyt      バイト・オフセット値を表示
000003D8 6100FE8E          bsr     chkcr
000003DC 66C4              bne     bend        キャリッジ・リターンが打たれた場合は,
000003DE 11400001          move.b  d0,1(a0)    オフセット値をメモリに書き込む
000003E2 60BE              bra     bend

                           *** goto subroutine ***  スーパバイザ・モードでサブルーチンへジャ
                                                    ンプ(gコマンドで起動する)
000003E4 6100FD38  gosub:  bsr     inspa0      アドレスの入力
000003E8 6100FE7E          bsr     chkcr       アドレスの後にキャリッジ・リターンが打た
000003EC 6602              bne     gend        れれば実行, ほかのキーでは実行しない
000003EE 4E90              jsr     (a0)
000003F0 4E75      gend:   rts

                           *** copy ***         メモリのブロック転送(cコマンドで起動する)
000003F2 6100FD48  copy:   bsr     inarea      オリジナルの先頭アドレス,終了アドレスを
000003F6 2448              movea.l a0,a2       入力
000003F8 6100FD24          bsr     inspa0      転送先のアドレスを入力
000003FC 6100FE6A          bsr     chkcr       最後にキャリッジ・リターンが打たれた場合
00000400 6702              beq     copy1       は実行, ほかのキーでは実行しない
00000402 6020              bra     cend
00000404 B1CA      copy1:  cmpa.l  a2,a0       転送先とオリジナルの先頭アドレスを比較
00000406 6408              bcc     copy3       し,転送先が大であればプリデクリメント・モ
                                               ードで転送する
00000408 30DA      copy2:  move.w  (a2)+,(a0)+
0000040A B3CA              cmpa.l  a2,a1
0000040C 64FA              bcc     copy2       ポスト・インクリメント・モードによる
0000040E 6014              bra     cend        転送
00000410 2009      copy3:  move.l  a1,d0
00000412 908A              sub.l   a2,d0
00000414 D088              add.l   a0,d0
00000416 5480              addq.l  #2,d0
00000418 2040              movea.l d0,a0       プリデクリメント・モードによる転送
0000041A 43E90002          lea     2(a1),a1
0000041E 3121      copy4:  move.w  -(a1),-(a0)
00000420 B3CA              cmpa.l  a2,a1
00000422 62FA              bhi     copy4
00000424 4E75      cend:   rts

                                                   メモリへ定数を書き込む
                           *** k-in constant write to memory ***(kコマンドで起動する)
00000426 6100FD14  kyin:   bsr     inarea      メモリ・エリアを入力
0000042A 6100FC8C          bsr     outsp
0000042E 6100FCDE          bsr     inword      定数(ワード)を入力
00000432 6100FE34          bsr     chkcr
```

## 〈リスト2-2〉モニタ（つづき）

```
478 00000436 6608                bne     kend        キャリッジ・リターンであれば実行
479 00000438 B1C9        klop:   cmpa.l  a1,a0
480 0000043A 6204                bhi     kend
481 0000043C 30C0                move.w  d0,(a0)+    定数の書き込み．書き込みはワード単位
482 0000043E 60F8                bra     klop
483 00000440 4E75        kend:   rts
484                                                  ユーザ・モードでユーザ・プログラムへジャンプ
485                      *** jump to user program *** （jコマンドで起動する）
486 00000442 6100FCDA    jmpuse: bsr     inspa0      アドレスの入力
487 00000446 6100FE20            bsr     chkcr       キャリッジ・リターンであれば実行
488 0000044A 660C                bne     useend
489 0000044C 40C0                move.w  sr,d0       スーパバイザ・フラグをクリア
490 0000044E 0880000D            bclr    #13,d0
491 00000452 2F08                move.l  a0,-(a7)    アドレスをプッシュ
492 00000454 3F00                move.w  d0,-(a7)    D0に転送されているステータス・レジスタをプッシュRTE 命令により，プログラムへジャンプ
493 00000456 4E73                rte
494 00000458 4E75        useend: rts
495
496                      *** output status registor format from d0 reg. ***
497 0000045A 48E7E000    outsr:  movem.l d0-d2,-(a7)  D0の下位1ワードをステータス・レジスタとして表示
498 0000045E 3200                move.w  d0,d1
499 00000460 E041                asr.w   #8,d1
500 00000462 3400                move.w  d0,d2
501 00000464 7054                moveq   #$54,d0     'T' トレース・ビット
502 00000466 613E                bsr     outbit
503 00000468 E341                asl.w   #1,d1
504 0000046A 7053                moveq   #$53,d0     'S' スーパバイザ・ビット
505 0000046C 6138                bsr     outbit
506 0000046E 3002                move.w  d2,d0
507 00000470 E040                asr.w   #8,d0
508 00000472 02000007    biwl:   andi.b  #7,d0       割り込みレベル
509 00000476 61000684            bsr     dlvnr
510 0000047A 3002                move.w  d2,d0
511 0000047C 6106                bsr     outccr
512 0000047E 4CDF0007            movem.l (a7)+,d0-d2
513 00000482 4E75                rts
514
515                      *** output ccr format  from d0 reg. ***  D0の下位1バイトをコンディション・コード・レジスタとして表示
516 00000484 48E7C000    outccr: movem.l d0/d1,-(a7)
517 00000488 1200                move.b  d0,d1
518 0000048A E741                asl     #3,d1
519 0000048C 7058                moveq   #$58,d0     'S' 拡張フラグ
520 0000048E 6116                bsr     outbit
521 00000490 704E                moveq   #$4e,d0     'N' ネガティブ・フラグ
522 00000492 6112                bsr     outbit
523 00000494 705A                moveq   #$5a,d0     'Z' ゼロ・フラグ
524 00000496 610E                bsr     outbit
525 00000498 7056                moveq   #$56,d0     'V' オーバフロー・フラグ
526 0000049A 610A                bsr     outbit
527 0000049C 7043                moveq   #$43,d0     'C' キャリ・フラグ
528 0000049E 6106                bsr     outbit
529 000004A0 4CDF0003            movem.l (a7)+,d0/d1
530 000004A4 4E75                rts
531
532 000004A6 E301        outbit: asl.b   #1,d1
533 000004A8 6502                bcs     bitskp
534 000004AA 702D                moveq   #$2d,d0     '-'
535 000004AC 600005B8    bitskp: bra     outchk
536
537                      *** monitor command input and controllor *** コマンド入力，制御ルーチン
538 000004B0 6100FC16    comman: bsr     outcr
539 000004B4 6100FC1C            bsr     outast      "*"を表示
540 000004B8 6100FC22            bsr     inecho      キー入力
541 000004BC 41FAFFF2            lea     comman(pc),a0  コマンド実行後のもどり番地をプッシュ
542 000004C0 2F08                move.l  a0,-(a7)
543
544 000004C2 0C00006F            cmpi.b  #'o',d0
545 000004C6 6700FEB6            beq     branch
546 000004CA 0C000067            cmpi.b  #'g',d0
547 000004CE 6700FF14            beq     gosub
548 000004D2 0C00006D            cmpi.b  #'m',d0
549 000004D6 6700FDD6            beq     memory
550 000004DA 0C000070            cmpi.b  #'p',d0
551 000004DE 6700FE42            beq     print
552 000004E2 0C000077            cmpi.b  #'w',d0
553 000004E6 6700FE68            beq     wide
554 000004EA 0C000072            cmpi.b  #'r',d0
555 000004EE 6700FD04            beq     outreg
556 000004F2 0C000074            cmpi.b  #'t',d0
557 000004F6 67000232            beq     trace
558 000004FA 0C000075            cmpi.b  #'u',d0
559 000004FE 67000234            beq     undisp
560 00000502 0C000071            cmpi.b  #'q',d0
561 00000506 67000210            beq     quick
562 0000050A 0C000063            cmpi.b  #'c',d0
563 0000050E 6700FEE2            beq     copy
564 00000512 0C00006B            cmpi.b  #'k',d0
565 00000516 6700FF0E            beq     kyin
566 0000051A 0C00006E            cmpi.b  #'n',d0
567 0000051E 67000222            beq     cont
568 00000522 0C000062            cmpi.b  #'b',d0
569 00000526 6712                beq     break
570 00000528 0C00006A            cmpi.b  #'j',d0
571 0000052C 6700FF14            beq     jmpuse
572 00000530 0C000010            cmpi.b  #$10,d0     contr-p プリント・フラグをセット
573 00000534 67000526            beq     prtset
574 00000538 4E75                rts
575
576                      *** break set and reset ***  ブレーク・ポイントのセットとリセット(bコマンドにより起動する)
577 0000053A 0839000700000006    break:  btst    #7,brkdis
578 00000542 6724                beq     brkoff
579 00000544 6100FBD8            bsr     inspa0      ブレーク・アドレスを入力
580 00000548 6100FD1E            bsr     chkcr       キャリッジ・リターンであれば実行
581 0000054C 665E                bne     brkend
582 0000054E 23C800000000        move.l  a0,brkad
583 00000554 33D000000004        move.w  (a0),brkir
584 0000055A 30BC4E4F            move.w  #$4e4f,(a0)     write trap #15
585 0000055E 08B9000700000006    bclr    #7,brkdis   ブレーク・アドレスにトラップ#15命令を書き込む
586 00000566 6044                bra     brkend
587
588 00000568 7013        brkoff: moveq   #$13,d0     "Break AD:"
589 0000056A 610000FC            bsr     emsgl       ブレーク・アドレスを表示
590 0000056E 203900000000        move.l  brkad,d0
591 00000574 0880001F            bclr    #31,d0      disenable
592 00000578 6100FD12            bsr     out3by
593 0000057C 7014                moveq   #$14,d0     " Inst."
594 0000057E 610000E8            bsr     emsgl       ブレーク・ポイントのインストラクションを表示
595 00000582 303900000004        move.w  brkir,d0
596 00000588 6100FAEE            bsr     outwor
597 0000058C 7015                moveq   #$15,d0     "Clear?"
598 0000058E 610000D8            bsr     emsgl       キャリジ・リターンが入力されれば，ブレーク・ポイントをクリアする
599 00000592 6100FCD4            bsr     chkcr
600 00000596 6614                bne     brkend
601 00000598 207900000000        movea.l brkad,a0
```

## 〈リスト2-2〉モニタ（つづき）

```
0000059E 30B900000004                 move.w  brkir,(a0)
000005A4 08F9000700000006             bset    #7,brkdis
000005AC 4E75                 brkend: rts

                              *** break entry point  ( trap #15 entry ) *** ブレークのエントリ・ルーチン
000005AE 48E78080             brkent: movem.l d0/a0,-(a0)
000005B2 6100FB14                     bsr     outcr
000005B6 7013                         moveq   #$13,d0          "Break AD:"
000005B8 610000AE                     bsr     emsgl
000005BC 202F000A                     move.l  $a(a7),d0
000005C0 5580                         subq.l  #2,d0
000005C2 6100FCC8                     bsr     out3by
000005C6 7002                         moveq   #2,d0
000005C8 6100009E                     bsr     emsgl
000005CC 302F0008                     move.w  8(a7),d0
000005D0 6100FE88                     bsr     outsr
000005D4 6100FAF2                     bsr     outcr
000005D8 4CDF0101                     movem.l (a7)+,d0/a0
000005DC 6100FC16                     bsr     outreg
000005E0 48E78080                     movem.l d0/a0,-(a7)
000005E4 7016                         moveq   #$16,d0          "Continue?"
000005E6 61000080                     bsr     emsgl
000005EA 6100FC7C                     bsr     chkcr
000005EE 6628                         bne     brkmon
000005F0 206F000A                     movea.l $a(a7),a0
000005F4 41E8FFFE                     lea     -2(a0),a0
000005F8 B1F900000000                 cmpa.l  brkad,a0
000005FE 661C                         bne     brktrp
00000600 30B900000004                 move.w  brkir,(a0)
00000606 2F48000A                     move.l  a0,$a(a7)
0000060A 08F9000700000006             bset    #7,brkdis
00000612 4CDF0101                     movem.l (a7)+,d0/a0
00000616 4E73                         rte
00000618 600000F6             brkmon: bra     sysent
0000061C 4CDF0101             brktrp: movem.l (a7)+,d0/a0
00000620 600000E4                     bra     extrap

                              *** exception vector initialize *** 例外処理ベクタのイニシャライズ.
00000624 41FA01FA             vecinz: lea     vectab(pc),a0      コールド・スタート時に実行される
00000628 327C0008                     movea.w #$0008,a1
0000062C 700A                         moveq   #10,d0
0000062E 22D8                 veclp1: move.l  (a0)+,(a1)+
00000630 5300                         subq.b  #1,d0
00000632 66FA                         bne     veclp1
00000634 327C0060                     movea.w #$60,a1
00000638 7018                         moveq   #24,d0
0000063A 22D8                 veclp2: move.l  (a0)+,(a1)+
0000063C 5300                         subq.b  #1,d0
0000063E 66FA                         bne     veclp2
00000640 4E71                         nop
00000642 4E71                         nop
00000644 4E75                         rts


                              *** exception message display-2 with "EXCEPTION"
00000646 48E74080             emsg2:  movem.l d1/a0,-(a7)        改行後"EXCEPTION"を表示
0000064A 41FA025C                     lea     msgdt(pc),a0
0000064E 6100FA78                     bsr     outcr
00000652 6100FAF8                     bsr     outmsg
00000656 1218                 emsg3:  move.b  (a0)+,d1
00000658 66FC                         bne     emsg3
0000065A 5300                         subq.b  #1,d0
0000065C 66F8                         bne     emsg3
0000065E 6100FAEC                     bsr     outmsg
00000662 4CDF0102                     movem.l (a7)+,d1/a0
00000666 4E75                         rts


                              *** exception message display-1 *** メッセージの表示.
                              emsg1:  movem.l d1/a0,-(a7)        メッセージ・ナンバはD0
00000668 48E74080                     lea     msgdt1(pc),a0
0000066C 41FA0245                     bra     emsg3
00000670 60E4


                              *** exception stack data display-1 *** バス・エラー,およびアドレス・エラー時でのスタックに積まれたデータの表示
00000672 2F00                 estac1: move.l  d0,-(a7)
00000674 6100FA52                     bsr     outcr
00000678 7001                         moveq   #1,d0
0000067A 61EC                         bsr     emsgl
0000067C 202F0016                     move.l  22(a7),d0
00000680 6100FC0A                     bsr     out3by
00000684 7002                         moveq   #2,d0            " SR:" ステータス・レジスタ
00000686 61E0                         bsr     emsgl
00000688 302F0014                     move.w  20(a7),d0
0000068C 6100FDCC                     bsr     outsr
00000690 7003                         moveq   #3,d0            " IR:" インストラクション・レジスタ
00000692 61D4                         bsr     emsgl
00000694 302F0012                     move.w  18(a7),d0
00000698 6100F9DE                     bsr     outwor
0000069C 7004                         moveq   #4,d0            " AA:" アクセス・アドレス
0000069E 61C8                         bsr     emsgl
000006A0 202F000E                     move.l  14(a7),d0
000006A4 6100F9E6                     bsr     outlon
000006A8 7005                         moveq   #5,d0            " SI:" スペシフィック・インフォメーション
000006AA 61BC                         bsr     emsgl            (R/Wフラグ,命令実行中,ファンクション・コード)
000006AC 302F000C                     move.w  12(a7),d0
000006B0 6100FB98                     bsr     outbin
000006B4 201F                         move.l  (a7)+,d0
000006B6 4E75                         rts


                              *** exception stack data display-2 *** 例外処理でのスタックに積まれたデータの表示
000006B8 2F00                 estac2: move.l  d0,-(a7)
000006BA 6100FA0C                     bsr     outcr
000006BE 7001                         moveq   #1,d0            " PC:" プログラム・カウンタ
000006C0 61A6                         bsr     emsgl
000006C2 202F000A                     move.l  $a(a7),d0
000006C6 6100FBC4                     bsr     out3by
000006CA 7002                         moveq   #2,d0            " SR:" ステータス・レジスタ
000006CC 619A                         bsr     emsgl
000006CE 302F0008                     move.w  8(a7),d0
000006D2 6100FD86                     bsr     outsr
000006D6 201F                         move.l  (a7)+,d0
000006D8 4E75                         rts


                              *** exception entry point ***


                              *** bus error entry point ***
000006DA 7006                 exbus:  moveq   #6,d0            " Bus Error !"
000006DC 6002                         bra     exad1


                              *** address error entry point ***
000006DE 7007                 exadr:  moveq   #7,d0            " Address Error !"
000006E0 61000424             exad1:  bsr     exad11
000006E4 602A                         bra     entend


                              *** illegal instruction entry point ***
```

〈リスト2-2〉モニタ（つづき）

```
000006E6 7008                  exill:  moveq   #8,d0           " Illegal Inst.."
000006E8 601E                          bra     extrl

                               *** zero divide entry point ***
000006EA 7009                  exzer:  moveq   #9,d0           " Zero Divide"
000006EC 601A                          bra     extrl

                               *** chk instruction entry point ***
000006EE 700A                  exchk:  moveq   #$a,d0          " CHK Inst.."
000006F0 6016                          bra     extrl

                               *** trapV instruction entry point ***
000006F2 700B                  extrv:  moveq   #$b,d0          " TRAPV Inst.."
000006F4 6012                          bra     extrl

                               *** privilege violation entry point ***
000006F6 700C                  expri:  moveq   #$c,d0          " Privilege"
000006F8 600E                          bra     extrl

                               *** line 1010 emulator entry point ***
000006FA 700D                  exl010: moveq   #$d,d0          " Line 1010"
000006FC 600A                          bra     extrl

                               *** line 1111 emulator entry point ***
000006FE 700E                  exl111: moveq   #$e,d0          " Line 1111 "
00000700 6006                          bra     extrl

                               *** interrupt autovector entry point ***
00000702 700F                  exint:  moveq   #$f,d0          " Interrupt At.."
00000704 6002                          bra     extrl

                               *** trap instruction entry point ***
00000706 7010                  extrap: moveq   #$10,d0         " TRAP Inst.."
00000708 6100FF3C              extrl:  bsr     emsg2
0000070C 61AA                          bsr     estac2
0000070E 4E71                          nop

                               *** stack intialize entry command input ***
                               entend:
00000710 3E7C0800              sysent: movea.w #stkadr,a7      initial stack address
00000714 6000FD9A                      bra     comman          スタックを初期値として,コマンド入力

                               *** trace command ***
                               *** quick trace ***  クイック・トレース（qコマンド）
00000718 7001                  quick:  moveq   #1,d0
0000071A 20790000000A                  movea.l trpc,a0
00000720 08F9000000000007              bset    #0,trcon
00000728 6038                          bra     cont2

                               *** trace ***            トレース（tコマンド）
0000072A 08F9000000000007      trace:  bset    #0,trcon
00000732 6008                          bra     undisl

                               *** undisplay trace ***  レジスタ表示なしのトレース(uコマンド)
00000734 08B9000000000007      undisp: bclr    #0,trcon
0000073C 6100F9E0              undisl: bsr     inspa0          トレース開始アドレスの入力
00000740 600E                          bra     contl

                               *** continue trace ***
00000742 08F9000000000007      cont:   bset    #0,trcon
0000074A 20790000000A                  movea.l trpc,a0
00000750 7011                  contl:  moveq   #$11,d0         " STEP"
00000752 6100FF14                      bsr     emsgl           ステップ数の入力
00000756 6100F9B6                      bsr     inword
0000075A 6100FB0C                      bsr     chkcr
0000075E 6702                          beq     cont2
00000760 4E75                          rts
00000762 23C80000000A          cont2:  move.l  a0,trpc
00000768 6100F94E                      bsr     outsp
0000076C 33C00000000E                  move.w  d0,trcoun
00000772 2F08                          move.l  a0,-(a7)
00000774 40C0                          move.w  sr,d0
00000776 103900000008                  move.b  trccr,d0        トレース・フラグのセット
0000077C 08C0000F                      bset    #15,d0          trace set
00000780 0880000D                      bclr    #13,d0          user mode ユーザ・モードとする
00000784 3F00                          move.w  d0,-(a7)
00000786 4CF97FFF00000010              movem.l trreg,d0-d7/a0-a6 セーブされていたレジスタをロードし,RTEにより実行
0000078E 4E73                          rte

                               *** trace entry point *** トレースのエントリ・ポイント
00000790 08170005              trent:  btst    #5,(a7)
00000794 6702                          beq     trentl
00000796 4E73                          rte
00000798 0839000000000007      trentl: btst    #0,trcon
000007A0 6750                          beq     trent2
000007A2 48E78080                      movem.l d0/a0,-(a7)
000007A6 6100F920                      bsr     outcr
000007AA 6100F91C                      bsr     outcr
000007AE 20790000000A                  movea.l trpc,a0
000007B4 7004                          moveq   #4,d0           プログラム・カウンタからのメモリ・データ 4ワードを表示
000007B6 6100F9C6                      bsr     outmb
000007BA 7011                          moveq   #$11,d0         " STEP"
000007BC 6100FEAA                      bsr     emsgl           ステップ数を表示
000007C0 30390000000E                  move.w  trcoun,d0
000007C6 6100F8B0                      bsr     outwor
000007CA 6100F8FC                      bsr     outcr
000007CE 7001                          moveq   #1,d0           " PC:"
000007D0 6100FE96                      bsr     emsgl           プログラム・カウンタを表示
000007D4 20390000000A                  move.l  trpc,d0
000007DA 6100FAB0                      bsr     out3by
000007DE 6100F8D8                      bsr     outsp
000007E2 302F0008                      move.w  8(a7),d0
000007E6 6100F8D0                      bsr     outsp
000007EA 4CDF0101                      movem.l (a7)+,d0/a0
000007EE 6100FA04                      bsr     outreg          レジスタ・ダンプ
000007F2 23EF00020000000A      trent2: move.l  2(a7),trpc
000007FA 6100F9BC                      bsr     chkbrk
000007FE 660A                          bne     trent3
00000800 53790000000E                  subq.w  #1,trcoun       トレース数が終了すればトレースを終了する
00000806 6702                          beq     trent3
00000808 4E73                          rte
0000080A 48F97FFF00000010      trent3: movem.l d0-d7/a0-a6,trreg
00000812 13EF000100000008              move.b  1(a7),trccr
0000081A 4FEF0006                      lea     6(a7),a7
0000081E 4E75                          rts

                               *** exception vector table ***
00000820 000006DA              vectab: .dc.l   exbus
00000824 000006DE                      .dc.l   exadr
00000828 000006E6                      .dc.l   exill
0000082C 000006EA                      .dc.l   exzer
00000830 000006EE                      .dc.l   exchk
```

それぞれの例外処理のエントリ・ポイント

```
850 00000834 000006F2                    .dc.l   extrv
851 00000838 000006F6                    .dc.l   expri
852 0000083C 00000790                    .dc.l   trent
853 00000840 000006FA                    .dc.l   ex1010
854 00000844 000006FE                    .dc.l   ex1111
855 00000848 00000702                    .dc.l   exint
856 0000084C 00000702                    .dc.l   exint   level1 intrpt
857 00000850 00000702                    .dc.l   exint   level2 intrpt
858 00000854 00000702                    .dc.l   exint   level3 intrpt
859 00000858 00000702                    .dc.l   exint   level4 intrpt
860 0000085C 00000702                    .dc.l   exint   level5 intrpt
861 00000860 00000702                    .dc.l   exint   level6 intrpt
862 00000864 00000702                    .dc.l   exint   level7 intrpt
863 00000868 00000706                    .dc.l   extrap  trap #0
864 0000086C 00000706                    .dc.l   extrap  trap #1
865 00000870 00000706                    .dc.l   extrap  trap #2
866 00000874 00000706                    .dc.l   extrap  trap #3
867 00000878 00000706                    .dc.l   extrap  trap #4
868 0000087C 00000706                    .dc.l   extrap  trap #5
869 00000880 00000706                    .dc.l   extrap  trap #6
870 00000884 00000706                    .dc.l   extrap  trap #7
871 00000888 00000706                    .dc.l   extrap  trap #8
872 0000088C 00000706                    .dc.l   extrap  trap #9
873 00000890 00000706                    .dc.l   extrap  trap #10
874 00000894 00000706                    .dc.l   extrap  trap #11
875 00000898 00000706                    .dc.l   extrap  trap #12
876 0000089C 00000706                    .dc.l   extrap  trap #13
877 000008A0 00000706                    .dc.l   extrap  trap #14
878 000008A4 000005AE                    .dc.l   brkent  trap #15 (break)
879
880
881
882                                 *** exception displaly codes ***
883 000008A8 455843455054494F       msgdt:  .dc.b   'EXCEPTION ',0
883 000008B0 4E2000
884 000008B3 00                       msgdtl: .dc.b   0
885 000008B4 50433A00                         .dc.b   'PC:',0         (#1)
886 000008B8 20535200                         .dc.b   ' SR',0         (#2)
887 000008BC 20495200                         .dc.b   ' IR',0         (#3)
888 000008C0 20414100                         .dc.b   ' AA',0         (#4)
889 000008C4 20534900                         .dc.b   ' SI',0         (#5)
890 000008C8 427573204572726F                 .dc.b   'Bus Error !',0 (#6)
890 000008D0 72202100
891 000008D4 4164647265737320                 .dc.b   'Address Error !',0     (#7)
891 000008DC 4572726F72202100
892 000008E4 496C6C6567616C20                 .dc.b   'Illegal Instruction',0 (#8)
892 000008EC 496E737472756374
892 000008F4 696F6E00
893 000008F8 5A65726F20446976                 .dc.b   'Zero Divide',0 (#9)
893 00000900 69646500
894 00000904 43484B20496E7374                 .dc.b   'CHK Instruction',0     (#a)
894 0000090C 72756374696F6E00
895 00000914 545241505620496E                 .dc.b   'TRAPV Instruction',0   (#b)
895 0000091C 737472756374696F
895 00000924 6E00
896 00000926 50726976696C6567                 .dc.b   'Privilege Violation',0 (#$c)
896 0000092E 652056696F6C6174
896 00000936 696F6E00
897 0000093A 4C696E6520313031                 .dc.b   'Line 1010 Emulator ',0 (#$d)
897 00000942 3020456D756C6174
897 0000094A 6F722000
898 0000094E 4C696E6520313131                 .dc.b   'Line 1111 Emulaotr ',0 (#$e)
898 00000956 3120456D756C616F
898 0000095E 74722000
899 00000962 496E746572727570                 .dc.b   'Interrupt Autovector ',0       (#$f)
899 0000096A 74204175746F7665
899 00000972 63746F722000
900 00000978 5452415020496E73                 .dc.b   'TRAP Instruction ',0   (#$10)
900 00000980 7472756374696F6E
900 00000988 2000
901 0000098A 205354455000                     .dc.b   ' STEP',0       (#$11)
902 00000990 0A0D2A2A2036384B                 .dc.b   $a,$d,'** 68K Monitor V-1.3 ***'
902 00000998 204D6F6E69746F72
902 000009A0 20562D312E33202A
902 000009A8 2A2A
903 000009AA 0A0D0A0D00                         .dc.b   $a,$d,$a,$d,0           (#$12)
904 000009AF 20427265616B2041                 .dc.b   ' Break AD:',0          (#$13)
904 000009B7 443A00
905 000009BA 20496E73743A00                     .dc.b   ' Inst:',0              (#$14)
906 000009C1 20436C6561723F00                 .dc.b   ' Clear?',0             (#$15)
907 000009C9 0A0D436F6E74696E                 .dc.b   $a,$d,'Continue? ',0    (#$16)
907 000009D1 75653F2000
908
909
910                                 *** input line editor ***
911 000009D6 347C0052               inline: movea.w #linbuf,a2
912 000009DA 6100F700               linelp: bsr     inecho
913 000009DE 0C000020                       cmpi.b  #$20,d0
914 000009E2 6416                           bcc     line1
915 000009E4 0C000008                       cmpi.b  #$08,d0
916 000009E8 660C                           bne     linend
917 000009EA B4FC0052                       cmpa.w  #linbuf,a2      'back'
918 000009EE 63E6                           bls     inline
919 000009F0 45EAFFFF                       lea     -1(a2),a2
920 000009F4 60E4                           bra     linelp
921 000009F6 4212                   linend: clr.b   (a2)
922 000009F8 4E75                           rts
923
924 000009FA 14C0                   line1:  move.b  d0,(a2)+
925 000009FC B4FC007A                       cmpa.w  #lbfend,a2
926 00000A00 65D8                           bcs     linelp
927 00000A02 60F2                           bra     linend
928
929
930                                 *** pit address definitions ***
931                                 pgcr    = pitadr        general control register
932                                 paddr   = pgcr+4        pa data direction register
933                                 pbddr   = paddr+2       pb data direction register
934                                 pcddr   = pbddr+2       pc data direction register
935                                 pivr    = pcddr+2       interrupt vector register
936                                 pacr    = pivr+2        pa control register
937                                 pbcr    = pacr+2        pb control register
938                                 padr    = pbcr+2        pa data register
939                                 pbdr    = padr+2        pb data register
940                                 psr     = pgcr+26       status register
941                                 ***
942                                 *** pit printer initialize ***
943 00000A04 13FC000000FFF001       inz2:   move.b  #0,pgcr
944 00000A0C 13FC00FF00FFF005               move.b  #$ff,paddr
945 00000A14 13FC000000FFF007               move.b  #0,pbddr
946 00000A1C 13FC006000FFF00D               move.b  #$60,pacr
947 00000A24 13FC00A000FFF00F               move.b  #$a0,pbcr
948 00000A2C 13FC003000FFF001               move.b  #$30,pgcr       activate port
949 00000A34 13FC00A800FFF00F               move.b  #$a8,pbcr
```

例外処理ベクタのデータ

メッセージ・データ

ターミナルからのテキスト入力．最大40文字とし，テキストは，$000452(linbuf)を先頭として格納される

パラレル・ポート/タイマのレジスタ・アドレスの定義

プリント出力のためのポートのイニシャライズ

〈リスト2-2〉モニタ（つづき）

```
00000A3C 13FC00A000FFF00F          move.b  #$a0,pbcr
00000A44 423900000050              clr.b   cpyflg
00000A4A 423900000051              clr.b   typerr
00000A50 08F9000700000006          bset    #7,brkdis
00000A58 600000B6                  bra     inzmes

                          *** copy to printer ***
00000A5C 0879000700000050 prtset: bchg    #7,cpyflg  プリント・フラグをセット.
                                                     コントロールpにより実行
00000A64 4E75                      rts

                          *** output character of d0 reg. *** ターミナル/プリンタへ1文字
                                                              出力. データはD0
00000A66 0839000700000050 outchk: btst    #7,cpyflg
00000A6E 672C                      beq     outvd      プリント・フラグ(cpyflg)がセットされてい
00000A70 613C                      bsr     type       ればターミナルとプリンタに出力
00000A72 08B9000700000051 perrck: bclr    #7,typerr
00000A7A 6720                      beq     outvd
00000A7C 2F08                      move.l  a0,-(a7)
00000A7E 41FA000C                  lea     prtmsg(pc),a0   prtmsg->a0
00000A82 6100F6C8                  bsr     outmsg
00000A86 205F                      movea.l (a7)+,a0
00000A88 6000FC86                  bra     sysent

00000A8C 0A0D5072696E7465 prtmsg: dc.b    $a,$d,'Printer Err !',0
00000A94 7220457272202100

                          *** out character to console *** ターミナルへ1文字出力
00000A9C 0839000100FFFF01 outvd:  btst    #1,aciac
00000AA4 67F6                      beq     outvd
00000AA6 13C000FFFF03              move.b  d0,aciad
00000AAC 4E75                      rts

                          *** print one character *** プリンタへ1文字出力
00000AAE 2F03             type:   move.l  d3,-(a7)
00000AB0 4283                      clr.l   d3
00000AB2 B6BC00200000     bsycnt: cmp.l   #$200000,d3
00000AB8 6724                      beq     lngbsy     出力ポートの書き込み可の待ち.
00000ABA 5283                      addq.l  #1,d3      時間制限があり,越えればプリン
00000ABC 0839000000FFF01B          btst    #0,psr     タ・エラーとする
00000AC4 67EC                      beq     bsycnt
00000AC6 13C000FFF011              move.b  d0,padr
00000ACC 13FC006800FFF00D          move.b  #$68,pacr       strobe ストローブ
00000AD4 13FC006000FFF00D          move.b  #$60,pacr
00000ADC 6010                      bra     typend
00000ADE 08F9000700000051 lngbsy: bset    #7,typerr
00000AE6 08B9000700000050          bclr    #7,cpyflg
00000AEE 261F             typend  move.l  (a7)+,d3
00000AF0 4E75                      rts

00000AF2 3200             outbx:  move.w  d0,d1
00000AF4 E040                      asr.w   #8,d0
00000AF6 6100F752                  bsr     outbin
00000AFA 4E75                      rts

00000AFC 06000030         dlvnr:  addi.b  #$30,d0
00000B00 6100FF64                  bsr     outchk
00000B04 4E75                      rts

00000B06 6100FB3E         exadll: bsr     emsg2
00000B0A 6100FB66                  bsr     estacl
00000B0E 4E75                      rts

00000B10 7012             inzmes  moveq   #$12,d0
00000B12 6100FB54                 bsr     emsgl
00000B16 6000F998                 bra     comman


00000000                          .bss

                          *** work area *** ワーク・エリア,実際のアドレスは$400～$452

00000000                  brkad:  .ds.l   1
00000004                  brkir:  .ds.w   1
00000006                  brkdis: .ds.b   1
00000007                  trcon:  .ds.b   1
00000008                  trccr:  .ds.b   1
0000000A                          .even
0000000A
0000000A                  trpc:   .ds.l   1
0000000E                  trcoun: .ds     1
00000010                  trreg:  .ds.l   16
00000050                  cpyflg: .ds.b   1
00000051                  typerr: .ds.b   1
00000052                  linbuf: .ds.b   40
                          lbfend: =       *


0000007A                          .end
```